

A. Asymptotic Analysis

Asymptotic upper bound: $f(n) = O(g(n))$ if there exist $c, n_0 > 1$ such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

Function hierarchy: $\log^a n < n^b < 2^{cn} < n! < n^{dn}$

For recursive algorithms, use **recurrence relations**

1. Come up with a recurrence relation for runtime $T(n)$.
2. Expand recurrence relation a few times, spot the pattern.
3. Figure out when it stops (base case).
4. Analyse the resulting sum.

Master theorem: Given $T(n) = aT(n/b) + cn^k$ and $T(1) = c$,

- $T(n) = \Theta(n^k)$ if $a < b^k$.
- $T(n) = \Theta(n^k \log n)$ if $a = b^k$.
- $T(n) = \Theta(n^{\log_b a})$ if $a > b^k$.

B. Searching

- B.1. Binary search** $O(\log n)$
1. Set indices $\ell = 1$ and $r = n$.
 2. While $\ell \leq r$, find middle index of sub-array.
 3. Compare middle element $A[\text{mid}]$ with k .
 - If $k = A[\text{mid}]$, found k . Return mid.
 - If $k > A[\text{mid}]$, then k lies in right half. Update $\ell = \text{mid} + 1$.
 - If $k < A[\text{mid}]$, then k lies in left half. Update $r = \text{mid} - 1$.
- $T(n) = T(n/2) + O(1) = O(\log n)$

- B.2. Partitioning** $O(n)$
1. Initially set indices $j = 1$ and $i = 1$. They mark boundaries of sub-arrays $A[2..j]$ and $A[j+1..i]$.
 2. Decide whether $A[i+1]$ belongs in the left or right sub-array.
 - If $A[i+1] < A[1]$, then $A[i+1]$ belongs in left sub-array. Swap $A[i+1]$ with $A[j+1]$. Increment i, j .
 - If $A[i+1] \geq A[1]$, then $A[i+1]$ belongs in right sub-array. Increment i .

- B.3. Quick select** $O(n^2)$
1. Set indices $\ell = 1, r = n$.
 2. Run partitioning on sub-array $A[\ell, r]$. Let i be the resulting index of the pivot.
 3. If $i = r$, then found r -th smallest integer. Return $A[i]$.
If $r < i$, then r -th smallest integer lies in left half. Update $\text{right} = i - 1$, recurse.
If $r > i$, then r -th smallest integer lies in the right half. Update $\text{left} = i + 1$, recurse.

- B.4. Randomised quick select** $O(n)$
1. Set indices $\ell = 1, r = n$.
 2. Keep randomly selecting an element in $A[\ell, r]$ to use as a pivot, run partition on $A[\ell, r]$ until pivot lies in middle third.
 3. If $i = k$, return $A[i]$.
If $k < i$, update $\text{right} = i$. Repeat step 2.
If $k > i$, update $\text{left} = i$. Repeat step 2.

- B.5. Peak finding** $O(\log n)$
1. Maintain a search interval $A[\ell, r]$.
 2. Probe the middle. If $A[\text{mid}]$ is a peak, return mid . Else recurse on left/right sub-array.

C. Sorting

Algorithm	Worst case	Expected	Space	Stability
Insertion sort	$O(n^2)$	$O(n^2)$	$O(1)$	stable
Selection sort	$O(n^2)$	$O(n^2)$	$O(1)$	unstable
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	stable
Quick sort	$O(n^2)$	$O(n^2)$	$O(n)$	unstable
Random quick sort	∞	$O(n \log n)$	$O(\log n)$	unstable

- C.1. Insertion sort** $O(n^2)$
- For $2 \leq i \leq n$, exchange $A[i]$ with entries in $A[1..i-1]$ until it is in place.
Invariant: At start of i -th iteration, $A[1..i-1]$ is in sorted order.

- C.2. Selection sort** $O(n^2)$
- For $1 \leq i \leq n$, find minimum element of $A[i..n]$, swap it with $A[i]$.
Invariant: At start of i -th iteration, $A[1..i-1]$ same as *sorted* $[1..i-1]$.

- C.3. Merge sort** $O(n \log n)$
- Divide array in half, recurse on two sub-arrays, merge.
 $T(n) = 2T(n/2) + O(n)$
Merge step takes in two sorted arrays, returns combined sorted array in $O(n)$.

- C.4. Quick sort** $O(n^2)$
- Partition array using any pivot, recurse on two sub-arrays, recombine.

- C.5. Randomised quick sort** *expected* $O(n \log n)$
- Random partition the array (so that final position of pivot lies in middle third), recurse on two sub-arrays, recombine.

- Heap sort** $O(n \log n)$
- Build min heap from array. While heap is non-empty, extract root node.

- Sort (a, b) by a then b :
1. Use any sorting algo to sort by b .
 2. Use stable sorting to sort by a (preserves ordering of b).

D. Elementary Data Structures

Array: read $O(1)$, insert $O(n)$, size $O(1)$
Linked list: set $O(n)$, delete $O(n)$, lookup $O(n)$, size $O(1)$, append $O(1)$
LIFO stack: push $O(1)$, pop $O(1)$, peek $O(1)$
FIFO queue: enqueue $O(1)$, dequeue $O(1)$, peek $O(1)$

Monotonic stack: Compare new element with top of stack. If order is violated, pop elements until order is restored, then push new element.
Max stack, queue: get max $O(1)$, add to all $O(1)$

- Augmenting data structures:
1. Choose an underlying data structure (array, linked list, tree)
 2. Determine additional info needed (weight, rank, height)
 3. Modify data structure to maintain additional info (summarisation: a node can summarise info in its subtree)
 4. Develop new operations

E. Binary Search Tree

- BST invariant:** all in left sub-tree $< \text{key} < \text{all in right sub-tree}$
- Search $O(h)$: If target $\leq \text{key}$, recurse left; else, recurse right.
 - Insertion $O(h)$: compare current node's key against k , insert into left/right subtree.
 - Deletion $O(h)$:
 - Leaf: delete
 - 1 child: delete, attach child to deleted node's parent.
 - 2 children: find successor of node to delete, swap the successor with the node to delete, delete the node (now leaf)
 - Tree traversals $O(n)$:
 - Pre-order: self, left, right
 - Post-order: left, right, self
 - In-order: left, self, right
 - Level-order: level by level

Size augmentation: size of node = size of left subtree + right subtree + 1

- Rank: check current node's key against k (rank of k equals number of elements smaller than k)
- Select: check current node's left subtree size against rank , recurse left/right

F. AVL Tree

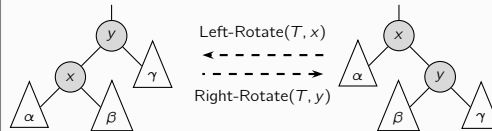
Organisation

AVL invariant: height of left and right sub-trees differ by at most 1
Height augmentation: height is number of edges on the longest path from root to leaf

- If node is leaf, height = 0
- If node is null, height = -1
- Else, height = $\max(\text{left.height}, \text{right.height}) + 1$

AVL tree with n nodes has height $O(\log n)$

Tree rotations



If one side is too tall, rotate it so that taller sub-tree moves up:

left heavy $\Rightarrow$ right rotation; right heavy $\Rightarrow$ left rotation

Rotation order:

- v is left-heavy
 - v .left is balanced: right rotation on v
 - v .left is left-heavy: right rotation on v
 - v .left is right-heavy: left rotation on v .left, right rotation on v
- v is right-heavy
 - v .right is balanced: left rotation on v
 - v .right is right-heavy: left rotation on v
 - v .right is left-heavy: right rotation on v .right, left rotate on v

G. Hash Table

- Organisation**
- n : **number of keys** currently inserted into the table
 - m : **table size** (i.e., size of the array)
 - **Hash function** $h: U \rightarrow \{0, 1, \dots, m-1\}$ maps key k to an integer. Element with key k is stored in slot $h(k)$.
 - General flow of inserting key k into hash table of size m :
 1. Compute the hash $h(k)$ (usually involves $\text{mod } m$).
 2. If the slot at that index is empty, insert x there.
 3. Otherwise, handle collisions.
 - **SUHA:** every key is equally likely to map to every slot, and keys are mapped independently
 - Uniform: $\mathbb{P}(h[k]) = \frac{1}{m}$
 - Independence: $\mathbb{P}(h[k_1] = i \mid h[k_2] = i) = \mathbb{P}(h[k_1] = i)$
- Combining these two properties, probability that any two distinct keys collide is $\frac{1}{m}$: $\forall k_1, k_2 \in U$ where $k_1 \neq k_2$, $\mathbb{P}(h[k_1] = h[k_2]) = \frac{1}{m}$

G.1. Separate chaining

Each slot contains a **linked list** of items, all items have same hash value
 Lookup, insert, delete: expected $O(n/m)$, worst $O(n)$

G.2. Open addressing

Each slot contains only one element; any collision will **probe** a sequence of slots until an empty one is found
 Lookup, insert, delete: expected $O(1)$, worst $O(n)$

G.3. Find most frequently occurring

1. Traverse the array once, maintain a hash table that records the frequency of each element encountered.
Key is the value of the element, value is the **frequency** of the element.
2. Retrieve all key-value pairs, find maximum frequency, then output all keys with that frequency.
Expected $O(n)$, worst-case $O(n^2)$ if all keys collide.

G.4. Find uniques

1. Traverse the array once, maintain a hash table that records the frequency of each element encountered. (see above)
2. Retrieve all key-value pairs, find keys with value = 1.

G.5. Longest sub-array with distinct elements

1. Maintain two pointers ℓ and r , and a hash table tracking elements in the current window $[\ell, r]$.
 2. Advance r by one. If not distinct, advance ℓ until all elements in $[\ell, r]$ are distinct again.
 3. Update the hash table to reflect the current window.
- Since we advance left at most n times, and advance right at most n times. We are making at most n lookup, insert and remove operations, so expected $O(n)$.

H. Binary Heap

Organisation

- A **binary heap** organises nodes in a complete binary tree, last level is filled from left to right
- **Min heap:** priority of parent $\leq$ children
- Binary heap with n nodes has height $O(\log n)$

Operations

- Insertion $O(\log n)$: insert at index $\text{size} + 1$, bubble up
- Peek min $O(1)$: return element at index 1
- Decrease key $O(\log n)$: locate node, update key, bubble up
- Extract min $O(\log n)$: swap root node with last node, remove last node, bubble down (swap with the *smaller* of two children)
- Heapify $O(n)$: apply bubble-down at indices $\text{size}/2, \dots, 1$

I. Graphs

	Adjacency matrix	Adjacency list	Edge list
Is-Adjacent	$O(1)$	$O(\deg(i))$	$O(m)$
Get-Neighbours	$O(n)$	$O(1)$	$O(m)$
In-Degree-Of	$O(n)$	$O(n+m)$	$O(m)$
Out-Degree-Of	$O(n)$	$O(1)$	$O(m)$
Space	$O(n^2)$	$O(n+m)$	$O(m)$

I.1. DFS

1. Start traversal at source vertex s . Mark s as visited.
2. For each neighbour of s , if not visited, **recurse** on it.
 - 1: $\text{visited} = \text{array of length } |V| \text{ full of false}$
 - 2: **procedure** DFT(G, s)
 - 3: $\text{visited}[s] = \text{true}$
 - 4: $\triangleright$ *Pre-order operations*
 - 5: **for** all v adjacent to s **do**
 - 6: $\quad \text{if } \text{visited}[v] == \text{false}$ **then** DFT(G, v)
 - 7: $\triangleright$ *Post-order operations*

I.2. BFS

1. Initialize a (FIFO) **queue**, stores the vertices to visit.
2. Mark s as visited, enqueue s .
3. While queue is not empty, dequeue x . For all neighbours of x , if not visited, enqueue, mark them as visited.
 - 1: **procedure** BFT(G, s)
 - 2: $Q = [s]$
 - 3: $\text{visited} = \text{array of length } V \text{ full of false}$
 - 4: $\text{visited}[s] = \text{true}$
 - 5: **while** $Q \neq \emptyset$ **do**
 - 6: $\quad x = \text{Dequeue}(Q)$
 - 7: $\quad \text{for all } y \text{ adjacent to } x \text{ do}$
 - 8: $\quad \quad \text{if } \text{visited}[y] == \text{false}$ **then**
 - 9: $\quad \quad \quad \text{Enqueue}(Q, y)$
 - 10: $\quad \quad \text{visited}[y] = \text{true}$

Connectedness

Input: undirected graph
 Pick any node and run BFS/DFS from it. Every other node must be reachable from that starting node.

s-t reachability

Input: directed graph
 Run BFT from s , check if t is visited; if yes, terminate and return true.

General reachability

Input: directed graph

1. Pick any node, run BFS/DFS from it, see if can reach every node.
2. Flip all the edges. Run BFS/DFS again.

Cycle detection in undirected graphs

Start BFS from unvisited nodes. If revisit a visited node, output true.

Graph colouring

Re-use same traversal algorithm as above (BFS that tries to visit every node in the graph, marks out every node in a connected component as visited), but keep a running id

I.3. Cycle detection in directed graphs $O(V + E)$ Observation: If we have visited a node, but then returned the recursion on it, then it is not part of a cycle. If we are within an active recursive call, and visited a node twice, then there is a cycle. - Start a DFS from source node s. Maintain a boolean array <i>in_stack</i> to record whether a node is currently in the recursion stack. - If s has been visited, return true if s is currently in recursion stack; false otherwise. - Otherwise, mark s as visited, and mark it as being in recursion stack. - For each neighbour of s, recurse on it. If any recursive call returns true, propagate true upwards. - After exploring all neighbours, pop s from the recursion stack.	All pairs shortest path (Floyd-Warshall algorithm) $O(V^3)$ Find shortest paths between every pair of vertices in a weighted graph	L.1. Kruskal's algorithm $O(E \log V)$ - Create a forest of single-vertex trees for each vertex. - Sort graph edges by weight. - Add cheapest edge to the forest (provided it does not form a cycle). Use UFDS to check if two vertices are in the same component. If not, add the edge into T, union the components containing u and v. - Stop when we have added $V - 1$ edges.	We treat the computation of a row as a sliding window maximum problem, using a max queue to determine the optimal choice of j. We use an Add-To-All operation to account for the $j \cdot v_i$ term as the window slides along the capacity axis. - For each item i, iterate through every possible remainder $k = 0, 1, \dots, w_i - 1$. - For a fixed remainder k, slide a window across the capacities $k, w_i + k, 2w_i + k, \dots, W$ and initialize a max queue to keep track of the optimal values within the current chain. - As the capacity multiplier m increases, apply an Add-To-All operation to the queue; this increments the potential value by v_i for each step forward, representing the addition of one more instance of item i. - Enqueue the value from the previous row $K[i - 1]$ at the current capacity and perform a dequeue if the queue size exceeds $x_i + 1$. This ensures that we never select more than x_i copies of the current item. - The maximum value in the max queue represents the best combination of a previous state's value and the accumulated value of the selected copies of item i, so assign it to $K(i, w)$.
I.4. Toposort Input: directed, acyclic graph - Maintain linked list T, to store all nodes seen so far in topo ordering. - Start traversal at source vertex s. Mark s as visited. - Retrieve the list of neighbours of vertex s. For each neighbour of s, if not visited, recurse on it. - After all neighbours of s have been processed, prepend s to T.	J.5. Problem solving - Modify existing algorithms directly. - Treat algorithms as a black box, but transform the input graph. Graph modelling: Many problems can be reduced to graph problems by: - Defining states as nodes. - Defining valid transitions as edges. - Running a suitable graph algorithm. <pre> graph LR Input -- Transform --> Graph Graph -- Algo --> Answer Answer -- Transform --> Solution </pre> Graph tricks: sometimes the graph is missing some state - Add virtual nodes/edges (e.g. add extra node to front/back), then run SSSP from those nodes - Duplicate nodes to represent state - copy graph into multiple layers - Reverse edges - Modify node weights e.g. log, reciprocal E.g. to force a path through P, duplicate graph, connect P to P' with weight 0, then run SSSP on $S \rightarrow D'$	L.2. Prim's algorithm $O(E \log V)$ - Initialize empty set T to store MST. Initialize priority queue to store edges currently under consideration. - Choose arbitrary starting vertex s. Mark it as in tree. For every neighbour v of s, insert the edge (s, v) into the priority queue, using edge weight as priority. - While pq is non-empty: extract edge (u, v) with smallest weight. - If both u and v are already in the tree, skip. Else, add edge to T, mark vertices as in tree. - Since v is a new addition to the tree, it brings in new potential edges. Examine all neighbors x of v and insert these new incident edges (v, x) into the priority queue Q. - Repeat until priority queue is empty.	Algorithm 1 Bounded Knapsack (Optimized) <pre> 1: Initialize $K[0 \dots n][0 \dots W]$ to 0 2: for $i = 1, \dots, n$ do 3: for $k = 0, \dots, w_i - 1$ do 4: $Q \leftarrow$ empty max queue 5: $m \leftarrow 0$ 6: while $m \cdot w_i + k \leq W$ do 7: Add-To-All(Q, v_i) 8: Enqueue($Q, K[i - 1, m \cdot w_i + k]$) 9: if $Q.size > x_i + 1$ then 10: Dequeue(Q) 11: $K[i, m \cdot w_i + k] \leftarrow \text{Max}(Q)$ 12: $m \leftarrow m + 1$ </pre>
J. Single Source Shortest Path	K. UFDS Represent sets by rooted trees; each node contains one member and each tree represents one set. In a disjoint-set forest, each member points only to its parent. The root of each tree contains the representative and is its own parent.	L.3. Boruvka's algorithm $O(E \log V)$ - Initialize all vertices as individual components. - Initialize MST as empty. - While > 1 component, do following for each component: Find the cheapest edge that connects this component to any other component. Add it to MST (if adding it does not form a cycle).	Since each state is enqueued and dequeued at most once per item, the total time complexity is reduced to $O(nW)$.
J.1. BFS $O(V + E)$ Input: directed, unweighted graph - Maintain distance array. Initialize (FIFO) queue. - Enqueue s into the queue. Mark s as visited. - While the queue is not empty, dequeue a node x. - For all neighbours of x, if not visited, mark as visited, update distance, enqueue.	J.2. Bellman-Ford $O(VE)$ Input: directed weighted graph without negative cycles Path relaxation: If $d[u] + w(u, v) < d[v]$, update $d[v] = d[u] + w(u, v)$ - Initially set $d[s] = 0$ and all other $d[v] = \infty$. - Relax all edges, for $V - 1$ iterations.	M. Dynamic Programming Spot DP problem: 1. overlapping subproblems 2. optimal substructure Template to approach DP problems: - Problem being solved - Valid subproblems - Contribution of each subproblem - Combine the subproblems Note: each problem should depend on constant number of subproblems	M.3. Longest common subsequence Let $L(i, j)$ be length of longest common subsequence between first i elements of A and first j elements of B. Base cases: $L(0, j) = 0, L(i, 0) = 0$ (empty sequence) Compare current element i from A and element j from B: - If they match, extend the previous longest common subsequence by 1 - If they don't match, take the maximum of either excluding the current element from A, or excluding it from B $L(i, j) = \begin{cases} \max \begin{cases} L(i - 1, j) \\ L(i, j - 1) \end{cases} & \text{if } A[i] \neq B[j] \\ L(i - 1, j - 1) + 1 & \text{if } A[i] = B[j] \end{cases}$
Negative cycle detection Run Bellman-Ford, then relax all the edges one more time (the V-th time). If there are changes, then negative cycle is present.	K.1. Heuristics Union by rank: attach shorter tree under taller one (rank is number of edges in longest simple path from node to a leaf) Path compression: whenever we find the root of a node, make all nodes along the path point directly to the root $\rightarrow$ flattens the tree	M.1. 0-1 knapsack $O(nW)$ Let $K(i, w)$ be the maximum value achievable using the first i items with combined weight w. Consider the last item (item i). - If $w_i > w$, exclude item i, consider only first $i - 1$ items - If $w_i \leq w$, we have two choices: exclude or include item i $K(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ K(i - 1, w) & \text{if } i > 0, w < w_i \\ \max \begin{cases} K(i - 1, w), \\ K(i - 1, w - w_i) + v_i \end{cases} & \text{if } i > 0, w \geq w_i \end{cases}$	M.4. Vertex cover on a tree Let $\text{cover}(r, \text{in_cover})$ represent the size of the vertex cover of the subtree rooted at r, where r is in the cover if <i>in_cover</i> = true. If r is in the cover, then for all children $c_1, \dots, c_t$ of r, they can be in or not in the cover, so take the minimum across both cases. Sum them up then + 1. $\text{cover}(r, \text{true}) = 1 + \sum_{i=1}^t \min(\text{cover}(c_i, \text{true}), \text{cover}(c_i, \text{false})).$
J.3. Toposort $O(V + E)$ Input: directed acyclic (DAG), potentially negative weights - Toposort the nodes - One pass of relaxation for all nodes	K.2. Operations - Make-Set(x): Creates a new set whose only member (and thus representative) is x. - Union(x, y): Merge the sets that contain x and y, say S_x and S_y, into a new set $S_x \cup S_y$ that is the union of these two sets. - In-Same-Set(x, y): Return whether x and y belong to the same set. - Find-Set(x): Returns a pointer to the representative of the (unique) set containing x. Runtime: $O(\alpha(n))$, where n is number of elements.	M.2. Unbounded knapsack $O(nW)$ Consider a binary choice for item i. - If $w < w_i$, ignore item i, consider only the first $i - 1$ items - If $w \geq w_i$, we have two choices: ignore, or take one copy of item i $K(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ K(i - 1, w) & \text{if } i > 0, w < w_i \\ \max \begin{cases} K(i - 1, w), \\ K(i, w - w_i) + v_i \end{cases} & \text{if } i > 0, w \geq w_i \end{cases}$	If r is not the cover, then for all children $c_1, \dots, c_t$ of r, we need to cover the edges $(r, c_1), \dots, (r, c_t)$, so all $c_1, \dots, c_t$ must be in the cover. Don't plus 1, since we're not including r here. $\text{cover}(r, \text{false}) = \sum_{i=1}^t \text{cover}(c_i, \text{true}).$
J.4. Dijkstra $O((V + E) \log V)$ Input: directed, non-negatively weighted graph Greedily visit the node with smallest distance estimate - Maintain a priority queue of nodes based on their distance estimates, so that we know which node has the smallest distance estimate so far. - Extract a node out of priority queue (using extract min), relax outgoing edges, thereby reducing distance estimates. - Repeat until priority queue is empty.	Connected components procedure Connected-Components(G) - for each vertex $v \in V$ do Make-Set(v) - for each edge $(u, v) \in E$ do - if Find-Set(u) $\neq$ Find-Set(v) then Union(u, v) procedure Same-Component(u, v) - return Find-Set(u) == Find-Set(v)	M.2.1. Bounded knapsack Item i has a limit of x_i subjects. For item i, choose to include it j times, where $0 \leq j \leq x_i$, and subject to constraint $j \cdot w_i \leq w$. $K(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ \max_{0 \leq j \leq \min\lfloor w/w_i \rfloor, x_i} (jv_i + K(i - 1, w - jw_i)) & \text{otherwise} \end{cases}$	Finally, take minimum across both cases above. Base case: if r is a leaf node, then $\text{cover}(r, \text{true}) = 1, \text{cover}(r, \text{false}) = 0$.
Multi-source BFS Create a super-source node, connect it to all source vertices with edges of weight 0. Then run BFS from the super-source node. When done, subtract all distances by 1.	L. Minimum Spanning Tree Input: connected, weighted undirected graph Output: spanning tree of minimum weight Properties of MST: - No cycles - If you cut a MST, the two pieces are MSTs Cycle property: In every cycle, the max weight edge is not in MST Cut property: For every cut, the lightest edge that crosses the cut is in MST (safe edge)	M.2.1. Bounded knapsack Item i has a limit of x_i subjects. For item i, choose to include it j times, where $0 \leq j \leq x_i$, and subject to constraint $j \cdot w_i \leq w$. $K(i, w) = \begin{cases} 0 & \text{if } i = 0 \\ \max_{0 \leq j \leq \min\lfloor w/w_i \rfloor, x_i} (jv_i + K(i - 1, w - jw_i)) & \text{otherwise} \end{cases}$	
Teleporting SSSP Construct a single cycle connecting all vertices in T, of cost 0. Run Dijkstra on modified graph.	Generic MST algorithm procedure Generic-MST(G) $A \leftarrow \emptyset$ - while A does not form a spanning tree do - Find a safe edge for A - Add e to A	Runtime $O(nW^2)$. To optimize, observe that for item i, the state $K(i, w)$ depends only on previous states $K(i - 1, w - j \cdot w_i)$, where the weights are all the same mod w_i.	
SSSP with expressways Can choose up to k edges whose weight becomes 0. - Create $k + 1$ copies of the graph. Duplicate each v as $v^{(0)}, v^{(1)}, \dots, v^{(k)}$, where index i represents exactly i free edges used. - For each edge (u, v), add edges $(u^{(i)}, v^{(i+1)})$ with weight 0. Run Dijkstra on $s^{(0)}$. The answer for each vertex v is $\min_{0 \leq i \leq k} \text{dist}(v_i)$.			